

Lecture 11: Q-Learning – How to Populate Q-table

Yaxin Bi
School of Computing
University of Ulster

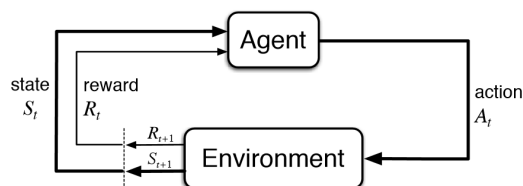
1

Contents

- ▶ Populating Q-table
- ▶ Hyperparameters
- ▶ Why Q-values Converge to Optimal Values
- ▶ Implementation of Q-Learning algorithm

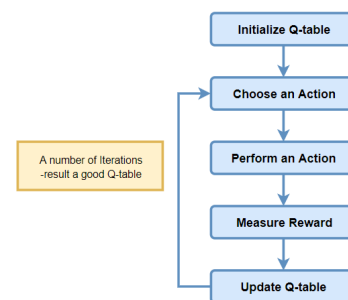
2

Architecture of Reinforcement Learning



3

Q-Learning Process



4

Q-Table

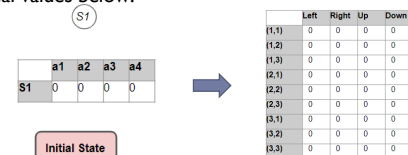
	State-Action Value (S x a)			
	Actions →			
	a_1	a_2	a_3	
s_1	Q_{11}	Q_{12}	Q_{13}	
s_2	Q_{21}	Q_{22}	Q_{23}	
s_3	Q_{31}	Q_{32}	Q_{33}	
s_4	Q_{41}	Q_{42}	Q_{43}	

$Q(s_3, a_3)$ i.e. Expected Return by taking Action a_3 from State s_3 and following Policy π after that

5

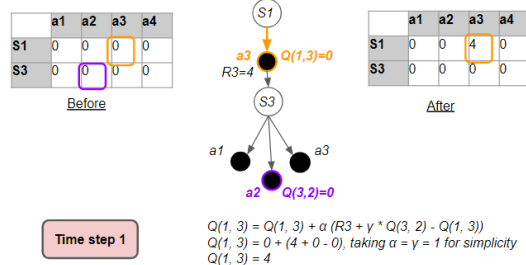
Populating Q-Table

- ▶ At the start of the game, the agent doesn't know which action is better than any other action.
- ▶ So the algorithm assigns all Q-values with arbitrary estimates such as all entries in the Q-table with 0
- ▶ As an illustration, how the Q-table gets populated with actual values below:



6

Populating Q-Table (cont'd)



7

Why Estimates Becoming More Accurate over Time-steps (part 1)

- ▶ A Q-learning algorithm starts with arbitrary estimates, and then at each time-step, it updates those estimates with other estimates.
- ▶ Why the algorithm converges to the Optimal Value?
 - ▶ because it gets updated with real observations
- ▶ The update formula combines three terms in some weighted proportion:
 - ▶ The reward for the current action
 - ▶ Best Estimated Q-value of the next state-action
 - ▶ Estimated Q-value of the current state-action

8

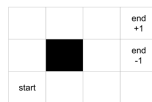
Accurate Values in the Formula

$$Q(S, A) = Q(S, A) + \alpha (R + \gamma \max_a Q(S', a) - Q(S, A))$$

Grounded in actual data

Estimate

Estimate



- ▶ The reward received is concrete data.
- ▶ Latter two estimates are not very accurate.
- ▶ The reward seems to play important role in improving the estimates based on actual experience with the environment.

9

Hyperparameters

- ▶ **Learning Rate:** α or α , can simply be defined as how much to accept the new value vs the old value, adjusting step size. The difference between new and old is also multiplied by the learning rate, which is then added to the previous Q-value.
- ▶ **Gamma:** γ or γ is a discount factor. It's used to balance immediate and future reward. Typically this value can range anywhere from 0.8 to 0.99.
- ▶ **Max:** $\max()$ takes the maximum of the future Q-values (reward) and applying it to this value for the current state. What this does is impact the **current action** by the possible **future reward**. The objective of allocating future reward to current actions is to help the agent select the highest return action at any given state.

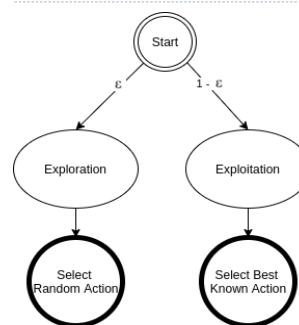
10

Epsilon-Greedy (ϵ)

- ▶ Epsilon greedy strategy play a vital role in trading off exploration and exploitation
- ▶ In the beginning, the epsilon rates will be higher. The agent will explore the environment and randomly choose actions as the agent does not know anything about the environment.
- ▶ As the agent explores the environment, progressively becoming more confident in estimating the Q-values. With decreasing of the epsilon rate, the agent starts to exploit the environment.
- ▶ The epsilon parameter introduces randomness into the algorithm, forcing the agent to try different actions. This helps not getting stuck in a local optimum (maxima).

11

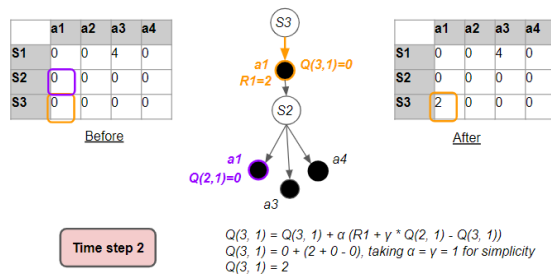
Exploration and Exploitation



- ▶ If epsilon is set to 0, the agent never explores but always exploits the knowledge it already has.
- ▶ On the contrary, setting the epsilon to 1 forces the algorithm to always take random actions and never use past knowledge.
- ▶ Usually, epsilon is selected as a small number close to 0.

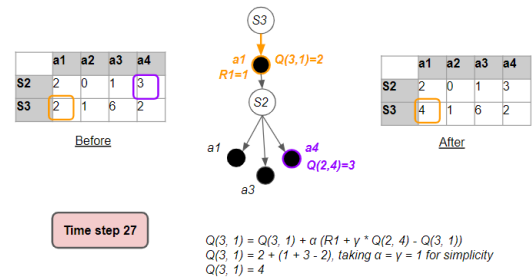
12

The Q-value Gets Updated with Real Rewards



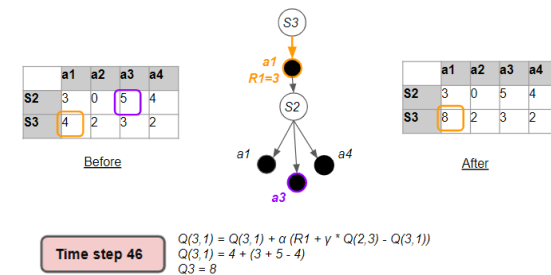
13

Second Visit to the State-Action Pair (S3, a1)



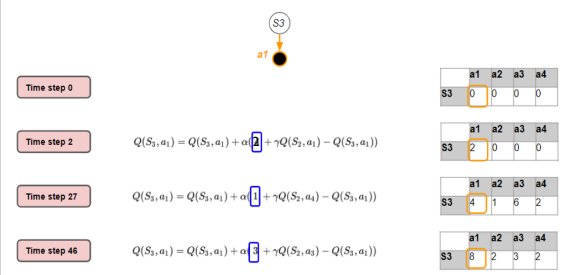
14

Third Visit to the State-Action Pair (S3, a1)



15

Put together, Progressing Updates over Time-Steps



16

Why Estimates Becoming More Accurate over Time-steps (part 2)

$$Q(S, A) = Q(S, A) + \alpha (R + \gamma \max_{a'} Q(S', a) - Q(S, A))$$

Grounded in actual data

Estimate

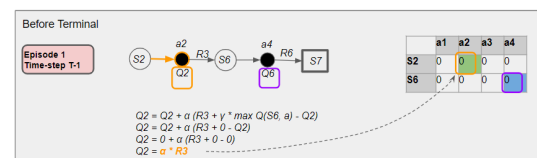
Estimate

- It has been seen how the Reward term converges towards the mean or expected value over many iterations
- What about the other two terms in the update formula which were Estimates and not actual data?
- How do they improve their accuracy through getting updated with real observations over time

17

Updating Q-table in (T-1)st time-step

- In the (T-1)st time-step that is towards the end of Episode 1, the agent picks an action as below. The Before-Terminal Q-value is updated based on the target action.

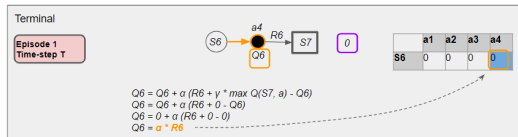


where Q2 and Q6 refer to Q(2, 2), and Q(6, 4), respectively

18

Updating Q-table in T^{th} time-step

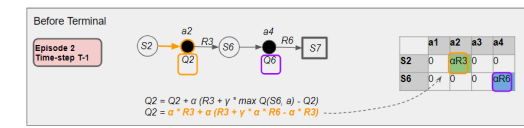
- ▶ In the T^{th} time-step of Episode 1, the agent picks an action to reach the next state which is a Terminal state.
- ▶ Since the next state is Terminal, there is no target action. So the 'max' term in the update formula is 0.
- ▶ The update to the Terminal Q-value is based solely on the actual reward data, and it does not rely on any estimated values.
- ▶ That causes the accuracy of the Terminal Q-value to improve. This ends the episode.



19

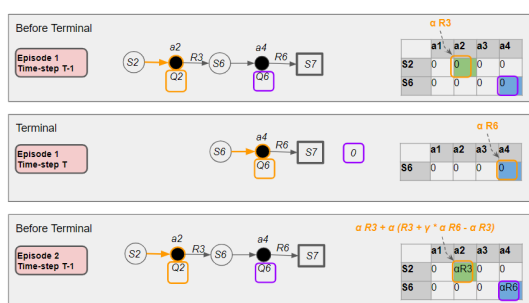
Updating in $(T-1)^{\text{st}}$ time-step of Episode 2

- ▶ Whenever visiting the Before-Terminal state again in a subsequent episode, for instance Episode 2, in the $(T-1)^{\text{st}}$ time-step, the Before-Terminal, Q-value is updated based on the target action as before.
- ▶ The 'max' term in the update formula corresponds to the Terminal Q-value.
- ▶ When the update happens, it is as though this Terminal Q-value gets transmitted backward to the Before-Terminal Q-value.



20

Put the Updates together over Time-steps



21

Two Estimated Q-value Terms Grounded

- ▶ As the accuracy of the Terminal Q-value slowly improves, the Before-Terminal Q-value becomes more accurate.
- ▶ Subsequently, those Q-Values backtrack the $(T-2)^{\text{th}}$ time-step and so on.
- ▶ In this way, as the estimated Q-values backtrack the path of the episode, the two estimated Q-value terms are also grounded in real observations with improving accuracy.

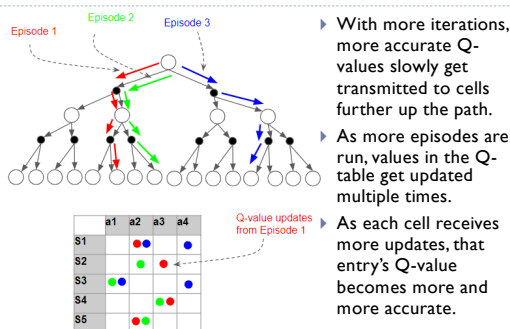
$$Q(S, A) = Q(S, A) + \alpha (R + \gamma \max_a Q(S', a) - Q(S, A))$$

Grounded in actual data

Get grounded in actual data as updates trickle back to earlier states

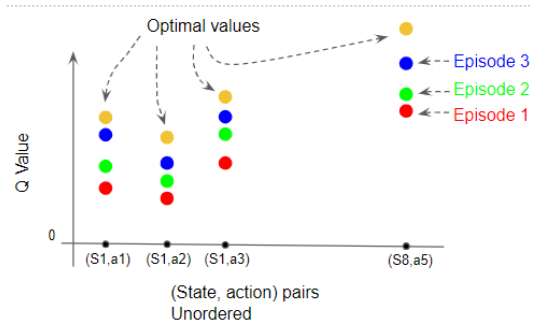
22

Illustration of Updates over Episodes



23

Optimal Values



24

Q-values Converge to Optimal Values

- ▶ The State-Action Value always depends on a policy. The Q-learning algorithm implicitly uses the ϵ -greedy policy to compute its Q-values.
- ▶ This policy encourages the agent to explore as many states and actions as possible. The more iterations it performs and the more paths it explores, the more confident the agent becomes that it has tried all the options available to find better Q-values.
- ▶ These are the two reasons why the ϵ -greedy policy algorithm eventually does find the optimal Q-values
 - ▶ With each iteration, the Q-values get better
 - ▶ If doing enough iterations, the agent will have evaluated all the possible options, and there will be no better Q-values that the agent can find.
- ▶ The above interpretation is informal but more formal mathematical proofs do exist

▶ 25

25

Summary of More Episodes More Accurate Q-values

- ▶ As doing more and more iterations, more accurate Q-values slowly get transmitted to cells further up the path. As each cell receives more updates, that cell's Q-value becomes more and more accurate.
- ▶ As more and more episodes are run, values in the Q-table get updated multiple times.
- ▶ The Q-values incrementally become more accurate with each update, moving closer and closer to the optimal values.
- ▶ This is not rigorous proof obviously, but gives us a gut feel for how Q-Learning works and why it converges.
- ▶ The later lectures will start to get to more interesting parts of Reinforcement Learning – introduction to Deep Q-Networks.

▶ 26

26

Implementation of gridWorld

```
def __init__(self):
    self.states = []
    self.actions = ["up", "down", "left", "right"]
    self.State = State()
    self.lr = 0.2
    self.exp_rate = 0.3

    # initial state reward
    self.state_values = {}
    for i in range(BOARD_ROWS):
        for j in range(BOARD_COLS):
            self.state_values[(i, j)] = 0 # set initial value to 0
```

▶ 27

27

Implementation of gridWorld_Q

```
def __init__(self):
    self.states = [] # record position and action taken at the position
    self.actions = ["up", "down", "left", "right"]
    self.State = State()
    self.isEnd = self.State.isEnd
    self.lr = 0.2
    self.exp_rate = 0.3
    self.decay_gamma = 0.9

    # initial Q values
    self.Q_values = {}
    for i in range(BOARD_ROWS):
        for j in range(BOARD_COLS):
            self.Q_values[(i, j)] = {}
            for a in self.actions:
                self.Q_values[(i, j)][a] = 0 # Q value is a dict of dict
```

▶ 28

28

Deterministic and Stochastic Policies (again)

- ▶ Deterministic policies are used in deterministic environments. In such environments, the actions taken determine the outcome. There is no uncertainty.
 - ▶ For instance, when you play chess and you move your pawn from A2 to A3, you're sure that your pawn will move to A3.
- ▶ On the other hand, a stochastic policy outputs a probability distribution over actions.
 - ▶ Instead of being sure of taking action a (for instance left), there is a probability of taking a different one (in this case 10% that the agent takes right).
 - ▶ The stochastic policy is used when the environment is uncertain. This process is often called a Partially Observable Markov Decision Process (POMDP).

▶ 29

29

Implementation of gridWorld_Q (cont'd)

```
if self.determine:
    if action == "up":
        nxtState = (self.state[0] - 1, self.state[1])
    elif action == "down":
        nxtState = (self.state[0] + 1, self.state[1])
    elif action == "left":
        nxtState = (self.state[0], self.state[1] - 1)
    else:
        nxtState = (self.state[0], self.state[1] + 1)
    self.determine = False
else:
    # non-deterministic
```

▶ 30

30

Implementation of gridWorld_Q (cont'd)

```

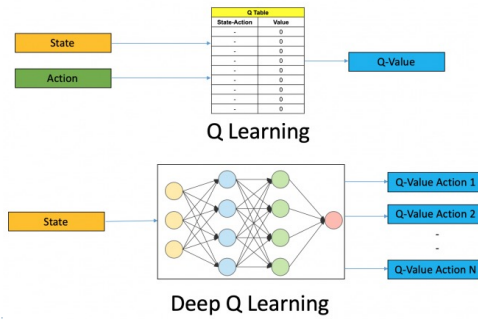
else:
    # non-deterministic
    action = self._chooseActionProb(action)
    self.determine = True
    nextState = self.nextPosition(action)

def _chooseActionProb(self, action):
    if action == "up":
        return np.random.choice(["up", "left", "right"], p=[0.8, 0.1, 0.1])
    if action == "down":
        return np.random.choice(["down", "left", "right"], p=[0.8, 0.1, 0.1])
    if action == "left":
        return np.random.choice(["left", "up", "down"], p=[0.8, 0.1, 0.1])
    if action == "right":
        return np.random.choice(["right", "up", "down"], p=[0.8, 0.1, 0.1])

```

31

A Glance of Deep Q Learning



32